

Reconocimiento de Elementos en el Desarrollo de un Programa Informático

Entornos de Desarrollo - 1º DAM

Alberto Florido Ramos



ÍNDICE

1. SELECCIÓN DE LA APLICACIÓN	2
2. LISTADO DE CARACTERÍSTICAS	3
3. DOCUMENTACIÓN INICIAL	4
4. ANÁLISIS DE REQUISITOS	5
5. INFORME DE LA TAREA Y REFLEXIÓN	6



1. SELECCIÓN DE LA APLICACIÓN

Un amigo mío es nutricionista, y ha solicitado ayuda para que hagamos una aplicación sobre nutrición y seguimiento de calorías y macronutrientes de los alimentos.

Check Your Macros, así se llamaría la app. Esta app reflejaría de forma exacta algunos conceptos:

- **Modelo de desarrollo ágil (Scrum).** Scrum permite trabajar en sprints de unos 15 días, con una funcionalidad nueva en cada uno, empezando por algo muy básico que se amplía después. Esto encaja con la app: la primera versión sólo registraría comidas y las siguientes añadirían estadísticas, objetivos o escáner de códigos de barras. Además, el mercado de apps de nutrición es competitivo, y llegar antes con un producto funcional es una ventaja.
- **Arquitectura de aplicaciones.** No es lo mismo un software que se ejecuta en un servidor y atiende a varios clientes que uno que corre en un móvil. *Check Your Macros* necesita ambos: una app móvil como cliente y un servidor con la base de datos de alimentos y las cuentas de usuario. Hay que pensar esa estructura desde el diseño.
- **Roles del desarrollo y usuarios.** La app tiene usuarios muy claros (el usuario final que registra sus comidas, el nutricionista que supervisa y el administrador), lo que facilita la fase de requisitos y de análisis.



2. LISTADO DE CARACTERÍSTICAS

Para tener más información de cómo funcionaría la aplicación, se ha hecho un listado de sus características (funcionales y no funcionales):

CARACTERÍSTICAS	TIPO
Registrar alimentos consumidos por comida (desayuno, comida, cena, snacks)	FUNCIONAL
Buscar alimentos en una base de datos	FUNCIONAL
Escanear el código de barras de un producto	FUNCIONAL
Calcular calorías y macronutrientes totales del día	FUNCIONAL
Fijar objetivos diarios de calorías y macros	FUNCIONAL
Consultar el historial y gráficas de progreso	FUNCIONAL
Crear cuenta e iniciar sesión	FUNCIONAL
Añadir alimentos personalizados o recetas propias	FUNCIONAL
Los datos personales se almacenan cifrados y con acceso protegido	NO FUNCIONAL (SEGURIDAD)
Registrar una comida en pocos toques	NO FUNCIONAL (USABILIDAD)

La diferencia está en que un requisito **funcional** describe *qué hace* el sistema (una acción concreta), mientras que uno **no funcional** describe *cómo debe comportarse* (una cualidad medible: rapidez, seguridad...).



3. DOCUMENTACIÓN INICIAL

Aquí se ha realizado una *ficha* para saber información general de la app:

- **Nombre:** Check Your Macros
- **Descripción breve:** aplicación móvil que permite registrar la alimentación diaria, calcular calorías y macronutrientes y seguir la evolución respecto a unos objetivos personales.
- **Objetivos:** ayudar a las personas a controlar lo que comen sin tener que hacer cálculos manuales ni consultar etiquetas una a una.
- **Usuarios principales (roles):**
 - *Usuario final:* registra sus comidas, fija objetivos y consulta su progreso.
 - *Nutricionista (opcional):* consulta el seguimiento de los usuarios que le den acceso.
 - *Administrador:* gestiona la base de datos y modera los alimentos añadidos por usuarios.
- **Plataformas:** móvil (iOS y Android) como principal, con posible versión web más adelante.
- **Modelo de desarrollo sugerido:** ágil, con Scrum, en sprints de unos 15 días. Se justifica porque los requisitos de una app de este tipo cambian con el feedback de los usuarios, y el tema señala que el modelo en cascada es lento y poco adecuado hoy, ya que no se pasa a la siguiente fase hasta terminar la anterior. En cambio, Scrum ofrece ciclos más cortos y predecibles.



4. ANÁLISIS DE REQUISITOS

A continuación, se mostrarán 2 tablas unidas en una sola. En una aparecen 5 requisitos funcionales, y en la otra otros 5 no funcionales, ambas con sus justificaciones:

REQUISITOS			
FUNCIONALES		NO FUNCIONALES	
REQUISITO	JUSTIFICACIÓN	REQUISITO	JUSTIFICACIÓN
El usuario podrá registrar alimentos en cada comida del día.	Es el núcleo del sistema. Al definirlo en la fase de requisitos, se traduce en el caso de uso principal del diagrama UML.	Seguridad: datos personales cifrados y acceso autenticado.	La app maneja datos de salud y hábitos personales. Un fallo aquí tiene un coste alto.
El sistema permitirá buscar alimentos por nombre y escanear su código de barras.	Reduce el esfuerzo del usuario. Es una funcionalidad ideal para un sprint propio, en línea con Scrum.	Usabilidad: registrar una comida en pocos toques.	Los usuarios abandonan las apps de seguimiento si son tediosas, y el público objetivo puede ser novato en nutrición.
El sistema calculará automáticamente calorías y macros del día.	Requiere una fase de análisis clara sobre cómo se relacionan alimentos en la base de datos.	Rendimiento: búsquedas en menos de 2 segundos.	Depende de la arquitectura de aplicaciones y se verifica en la fase de pruebas.
El usuario podrá definir objetivos diarios y ver su cumplimiento.	Aporta valor de seguimiento. Se puede incorporar en un sprint posterior a la versión básica.	Compatibilidad: funcionamiento en iOS y Android.	Condiciona la fase de diseño, porque hay que identificar los recursos y tecnologías necesarios.
El usuario podrá crear su cuenta, iniciar sesión y consultar su historial.	Sin cuentas no hay datos persistentes. Implica al arquitecto de software, decide cómo se comunican el móvil y el servidor.	Mantenibilidad: código modular con documentación.	sin documentación sería muy difícil corregirlo. Se pueden usar patrones de diseño para lograr una estructura ordenada.



5. INFORME DE LA TAREA Y REFLEXIÓN

Este trabajo ha sido realizado por Alberto Florido Ramos (yo). He asumido todos los roles descritos en el tema: como **analista de sistema**, definí las necesidades y los requisitos de la aplicación, como **diseñador** planteé su estructura y sus funcionalidades, y como **arquitecto de software** decidí la relación entre la app móvil y el servidor. También he redactado la documentación.

En cuanto a la reflexión, la fase que más me ha costado ha sido la de **requisitos**. Es una de las más tediosas porque el cliente suele cambiar de opinión, y en mi caso he sido a la vez cliente y desarrollador. Me di cuenta de que muchos requisitos que parecían claros eran ambiguos, como el nivel de precisión que debería tener el cálculo de macros.

También me ha resultado complicada la fase de **diseño**, porque exige pensar en la arquitectura antes de escribir código y, en una app de nutrición, hay que decidir cómo se conectan el móvil, el servidor y la base de datos de alimentos.

Sobre los modelos de desarrollo, el **modelo en cascada** me habría obligado a cerrar todos los requisitos desde el principio, algo poco realista. El **modelo en espiral** me parece más flexible, pero con ciclos de duración variable. **Scrum** me ha parecido el más práctico, porque un producto pequeño en sprints cortos me permite verificar cada funcionalidad antes de avanzar. Aun así, trabajando solo, Scrum pierde parte de su sentido colaborativo, y tendría que adaptarlo.